

IHK Algorithmus – Prüfungsaufgaben nach Grundmustern

Wichtig Deutsch فارسی

Für jedes Grundmuster findest du hier 1–2 typische Aufgaben im Stil der IHK-Prüfung (vereinfachte bzw. sinngemäße Versionen) mit vollständiger Lösung im Pseudocode (IHK-Stil).

برای هر الگوی اصلی، ۱–۲ مثال شبیه سؤال‌های واقعی IHK (نسخه ساده شده و «هم‌معنی») با جواب کامل به سبک شبه‌کد IHK داده شده است. کنار هر سؤال توضیح فارسی و ترفند حل هم نوشته شده تا در ذهن بماند.

Muster 1 – Suche in Daten (Rückgabe boolean / Index / Objekt)

Aufgabe 1.1 – Termin in Arztpraxis prüfen (getAppointment)

Eine Gemeinschaftspraxis verwaltet Termine für mehrere Ärzte. Für jeden Arzt liegt ein Terminplan als zweidimensionales Integer-Array vor. Jede Zeile enthält: Monat, Tag und die Belegung von acht Zeitslots (0 = frei, 1 = belegt). Über die Funktion `getTimeTable(id: Integer) → 2-dim-Integer-Array` erhältst du den Terminplan für einen Arzt.

Implementiere die Funktion `getAppointment(month: Integer, day: Integer, id: Integer, slot: Integer) → boolean`. Sie soll **true** zurückgeben, falls für den Arzt mit der Nummer `id` am angegebenen Datum ein Termin möglich ist, sonst **false**. Wenn `slot = -1`, reicht ein beliebiger freier Slot an diesem Tag.

یک درمانگاه برای هر داکتر یک جدول دوبعدی دارد: ستون‌های اول ماه و روز، بقیه ستون‌ها ۸ «Slot» در روز؛ یعنی خالی و ۱ یعنی پر. تابع `getTimeTable(id)` این جدول را می‌دهد. باید متد `getAppointment(...)` را بنویسی که اگر در آن روز برای این داکتر حداقل یک وقت آزاد مطابق شرط وجود داشت، **true** برگرداند وگرنه **false**. اگر `slot = -1` است، هر `slot` خالی قبول است، اگر عدد است همان `slot` باید خالی باشد.

```
Funktion getAppointment(  
    month: Integer,  
    day: Integer,  
    id: Integer,  
    slot: Integer  
) → boolean  
    plan = getTimeTable(id)           // 2-dim-Array für diesen Arzt  
    zeilen = anzahlZeilen(plan)  
  
    für i = 0 bis zeilen - 1  
        wenn plan[i][0] = month  
            und plan[i][1] = day dann  
  
                // Fall 1: bestimmter Slot  
                wenn slot ≠ -1 dann  
                    spalte = 2 + slot  
                    wenn plan[i][spalte] = 0 dann
```

```

        return true          // dieser Slot frei
    sonst
        return false        // Tag gefunden, Slot aber belegt
    end wenn

// Fall 2: beliebiger freier Slot reicht
sonst
    für s = 2 bis 9          // alle 8 Slots prüfen
        wenn plan[i][s] = 0 dann
            return true     // irgendein Slot frei
        end wenn
    end für
    return false            // Tag gefunden, kein Slot frei
end wenn
end wenn
end für

// kein Eintrag für Monat/Tag gefunden
return false
end Funktion

```

Tricks (Suche + Sonderfall Slot = -1)

- Sofort zurückgeben, sobald passende Zeile gefunden ist.
- Spezialfall slot = -1 im Inneren behandeln, nicht mit komplizierten Bedingungen mischen.
- به محض پیدا شدن روز درست تصمیم بگیر؛ یا همان slot را چک کن، یا اگر -1 است تمام slotها را ببین و نتیجه را همان جا برگردان.

Aufgabe 1.2 – Suche nach Top-Seller (sucheTopSeller)

Ein Online-Shop speichert beliebig viele Artikel in einem Array `artikelListe`. Für einen Artikel kann über die Funktion `ermittleWert(artikel: Artikel, kriterium: Integer) → Integer` ein Kennwert (z. B. Umsatz, Anzahl Verkäufe, Bewertung) ermittelt werden.

Implementiere die Funktion `sucheTopSeller(kriterium: Integer, vorgabewert: Integer) → String`. Sie soll die Artikelnummer des Artikels mit dem höchsten Kennwert zurückgeben, der mindestens den vorgabewert erreicht. Falls kein Artikel den Vorgabewert erreicht, soll ein leerer String zurückgegeben werden.

یک فروشگاه آنلاین آرایه‌ای از Artikel دارد. تابع `ermittleWert(...)` برای هر Artikel یک «ارزش» برمی‌گرداند. باید متدی بنویسی که شماره آن Artikel را برگرداند که هم حداقل به مقدار `vorgabewert` رسیده باشد و هم بین این‌ها بیشترین مقدار را داشته باشد. اگر هیچ‌کدام شرط را نداشتند، رشته خالی بده.

```

Funktion sucheTopSeller(
    kriterium: Integer,
    vorgabewert: Integer
) → String
    besterWert = -999999
    besteId = ""
    n = länge(artikelListe)

    für i = 0 bis n - 1

```

```

wert = ermittleWert(artikelListe[i], kriterium)
wenn wert ≥ vorgabewert
    und wert > besterWert dann
        besterWert = wert
        besteId = artikelListe[i].nummer
    end wenn
end für

return besteId    // "" falls keiner geeignet
end Funktion

```

ترفند: وقتی باید «بهترین» را پیدا کنی و شرط حداقل هم داری، معمولاً یک متغیر برای بهترین مقدار و یک متغیر برای «شناسهٔ بهترین» نگه می‌داری و فقط وقتی مقدار جدید هم شرط حداقل را داشت و هم از best بیشتر بود، آن‌ها را آپدیت می‌کنی.

Muster 2 – Zählen mit Bedingung

Aufgabe 2.1 – Anzahl Kunden über Limit

Ein Unternehmen speichert Kunden in einem Array `kunden: Array<Kunde>`. Jeder Kunde hat das Attribut `umsatz: Real`. Schreibe eine Funktion, die ermittelt, wie viele Kunden einen Umsatz größer oder gleich einem vorgegebenen Limit haben.

آرایه‌ای از `Kunden` داریم و هر `Kunde` یک فیلد `umsatz` دارد. باید تعداد مشتریانی را بشماریم که فروش آن‌ها حداقل برابر یک `limit` است.

```

Funktion anzahlKundenMitLimit(
    kunden: Array<Kunde>,
    limit: Real
) → Integer
n = länge(kunden)
anzahl = 0

für i = 0 bis n - 1
    wenn kunden[i].umsatz ≥ limit dann
        anzahl = anzahl + 1
    end wenn
end für

return anzahl
end Funktion

```

Aufgabe 2.2 – Freie Plätze im Kurs zählen

In einem Schulungssystem wird für einen Kurs ein Array `plaetze: Array<boolean>` geführt (`true` = belegt, `false` = frei). Schreibe eine Funktion, die die Anzahl freier Plätze zurückgibt.

آرایه‌ای از `boolean` داریم که برای هر صندلی کلاس مشخص می‌کند پر است یا خالی. باید تعداد صندلی‌های خالی را بشماریم.

```

Funktion zaehleFreiePlaetze(plaetze: Array<boolean>) → Integer
  n = länge(plaetze)
  anzahl = 0
  für i = 0 bis n - 1
    wenn plaetze[i] = false dann
      anzahl = anzahl + 1
    end wenn
  end für
  return anzahl
end Funktion

```

Muster 3 – Minimum / Maximum / Durchschnitt

Aufgabe 3.1 – Kleinster und größter Messwert

Ein Sensor liefert Messwerte, die in einem Integer-Array gespeichert werden. Implementiere zwei Funktionen, die Minimal- und Maximalwert berechnen.

آرایه‌ای از مقادیر Integer داریم (داده‌های سنسور). باید دو متد بنویسی که کمترین و بیشترین مقدار را برگردانند.

```

Funktion findeMinimum(W: Array<Integer>) → Integer
  min = W[0]
  für i = 1 bis länge(W) - 1
    wenn W[i] < min dann
      min = W[i]
    end wenn
  end für
  return min
end Funktion

```

```

Funktion findeMaximum(W: Array<Integer>) → Integer
  max = W[0]
  für i = 1 bis länge(W) - 1
    wenn W[i] > max dann
      max = W[i]
    end wenn
  end für
  return max
end Funktion

```

Aufgabe 3.2 – Durchschnittliche Bearbeitungszeit

Die Bearbeitungszeiten von Support-Tickets werden in Minuten als Array<Integer> gespeichert. Implementiere eine Funktion, die die durchschnittliche Bearbeitungszeit (Real) zurückgibt. Wenn keine Tickets vorhanden sind (Array-Länge 0), soll 0.0 zurückgegeben werden.

آرایه‌ای از مدت زمان رسیدگی به تیکت‌ها داریم. باید میانگین را حساب کنی؛ اگر آرایه خالی بود، مقدار ۰٫۰ را برگردان.

```

Funktion durchschnittZeit(zeiten: Array<Integer>) → Real
    wenn länge(zeiten) = 0 dann
        return 0.0
    end wenn

    summe = 0
    für i = 0 bis länge(zeiten) - 1
        summe = summe + zeiten[i]
    end für

    return summe / länge(zeiten)
end Funktion

```

Muster 4 – Liste aufbauen / filtern

Aufgabe 4.1 – Mitarbeiter mit Mindestlohn in Liste sammeln

Eine Firma speichert ihre Mitarbeiter in einem Array `mitarbeiter: Array<Mitarbeiter>`. Jeder Mitarbeiter hat die Attribute `stunden` und `stundenlohn`. Implementiere eine Funktion, die eine Liste aller Mitarbeiter zurückgibt, deren monatlicher Lohn (`stunden * stundenlohn`) größer oder gleich einem vorgegebenen Mindestlohn ist.

آرایه‌ای از `Mitarbeiter` داریم و برای هر کدام ساعت کار و دستمزد ساعتی مشخص است. باید لیستی از آن کارمندانی بسازی که مجموع حقوق‌شان حداقل برابر `mindestLohn` است.

```

Funktion filterMitarbeiter(
    mitarbeiter: Array<Mitarbeiter>,
    mindestLohn: Real
) → Liste<Mitarbeiter>
    liste = neueListe()
    n = länge(mitarbeiter)

    für i = 0 bis n - 1
        lohn = mitarbeiter[i].stunden * mitarbeiter[i].stundenlohn
        wenn lohn ≥ mindestLohn dann
            liste.hinzufuegen(mitarbeiter[i])
        end wenn
    end für

    return liste
end Funktion

```

Aufgabe 4.2 – Produkte mit Lagerbestand in Bestellliste übernehmen

In einem Warenwirtschaftssystem werden Produkte in einem Array `produkte: Array<Produkt>` gespeichert. Jedes Produkt hat das Attribut `bestand: Integer`. Schreibe eine Funktion, die eine Liste aller Produkte zurückgibt, deren Bestand `> 0` ist.

آرایه‌ای از Produkt داریم و هر کدام یک bestand (موجودی) دارد. باید لیستی از محصولات برگردانی که موجودی‌شان بیشتر از صفر است.

```
Funktion baueBestellListe(produkte: Array<Produkt>) → Liste<Produkt>
  liste = neueListe()
  n = länge(produkte)
  für i = 0 bis n - 1
    wenn produkte[i].bestand > 0 dann
      liste.hinzufuegen(produkte[i])
    end wenn
  end für
  return liste
end Funktion
```

Muster 5 – Vergleichs- und Sortiermethoden

Aufgabe 5.1 – Mitarbeiter nach Gehalt vergleichen

In einer Sortierfunktion sollen Mitarbeiter nach ihrem Gehalt sortiert werden. Implementiere die Vergleichsfunktion `vergleicheMitarbeiter(m1: Mitarbeiter, m2: Mitarbeiter) → Integer`, die `-1` zurückgibt, wenn `m1` weniger verdient als `m2`, `1` wenn `m1` mehr verdient, und `0` bei gleichem Gehalt.

برای مرتب‌سازی Mitarbeiter به تابع کمکی نیاز داریم تا دو کارمند را بر اساس حقوق مقایسه کند. باید مطابق قرارداد برابر/کمتر/بیشتر مقدار `-1` یا `0` یا `1` را برگرداند.

```
Funktion vergleicheMitarbeiter(
  m1: Mitarbeiter,
  m2: Mitarbeiter
) → Integer
  wenn m1.gehalt < m2.gehalt dann
    return -1
  end wenn

  wenn m1.gehalt > m2.gehalt dann
    return 1
  end wenn

  return 0
end Funktion
```

Aufgabe 5.2 – Räume nach Kapazität sortieren

Eine Schule verwaltet Räume in einem Array `raeume: Array<Raum>`. Jeder Raum hat das Attribut `plaetze: Integer`. Implementiere eine Prozedur `sort(raeume: Array von Raum)`, die das Array nach aufsteigender Plätzezahl sortiert (Bubble-Sort-ähnlich).

آرایه‌ای از Raum داریم و هر کلاس تعداد صندلی‌ها را در `plaetze` ذخیره کرده است. باید آرایه را به صورت صعودی نسبت به این تعداد مرتب کنی (الگوی Bubble Sort).

```

Prozedur sort(raeume: Array von Raum)
  n = länge(raeume)
  für i = 0 bis n - 2
    für j = 0 bis n - 2 - i
      wenn raeume[j].plaetze > raeume[j+1].plaetze dann
        temp = raeume[j]
        raeume[j] = raeume[j+1]
        raeume[j+1] = temp
      end wenn
    end für
  end für
end Prozedur

```

Muster 6 – OOP / Strategy / Delegation (z. B. getVersandkosten)

Aufgabe 6.1 – Versandkosten über Interface berechnen

Gegeben ist das Interface Versand mit der Methode berechnen(w: Warenkorb): double. Die Klasse Warenkorb besitzt das Attribut versand: Versand. Implementiere in der Klasse Warenkorb die Methode getVersandkosten(): double, die die Versandkosten für den aktuellen Warenkorb zurückgibt.

اینترفیس Versand متدی دارد به نام berechnen(Warenkorb). کلاس Warenkorb یک فیلد versand از این نوع دارد. باید متد getVersandkosten() را در Warenkorb پیاده‌سازی کنی که کار را به این آبجکت بسپارد.

```

Klasse Warenkorb
  // Attribute (vereinfacht):
  artikel : Artikel[]
  anzahl  : Integer[]
  versand : Versand

  Methode getVersandkosten() : double
    return versand.berechnen(this)
  end Methode
Ende Klasse

```

نکته: در چنین سؤال‌هایی بدنه متد معمولاً فقط همین یک خط است. کافی است this (همین Warenkorb فعلی) را به استراتژی versand بدهی.

Muster 7 – Umrechnung / Zahlen-Konvertierung

Aufgabe 7.1 – Dezimalzahl in Binärzahl umrechnen

Implementiere die Funktion Umrechnung(dezimalzahl: int) → String, die eine positive Dezimalzahl in eine Binärdarstellung als String umrechnet. Für den Wert 0 soll der String "0" zurückgegeben werden.

متدی بنویس که یک عدد صحیح مثبت را به رشته باینری تبدیل کند. اگر عدد ۰ بود، رشته "0" را برگرداند.

```
Funktion Umrechnung(dezimalzahl: Integer) → String
  wenn dezimalzahl = 0 dann
    return "0"
  end wenn

  ergebnis = ""
  zahl = dezimalzahl

  solange zahl > 0
    rest = zahl mod 2
    ergebnis = "" + rest + ergebnis  // vorne anhängen
    zahl = zahl / 2
  end solange

  return ergebnis
end Funktion
```

Aufgabe 7.2 – Sekunden in Stunden:Minuten:Sekunden

Implementiere die Funktion `formatZeit(sekunden: Integer) → String`, die eine Zeitspanne in Sekunden in das Format "HH:MM:SS" umrechnet (mit führenden Nullen).

متدی بنویس که تعداد ثانیه‌ها را به رشته "HH:MM:SS" تبدیل کند (با صفرهای ابتدای عدد).

```
Funktion formatZeit(sekunden: Integer) → String
  stunden = sekunden / 3600
  rest = sekunden mod 3600
  minuten = rest / 60
  sek = rest mod 60

  hh = formatZweiZiffern(stunden)
  mm = formatZweiZiffern(minuten)
  ss = formatZweiZiffern(sek)

  return hh + ":" + mm + ":" + ss
end Funktion
```

تابع فرضی `formatZweiZiffern(x)` عدد را به رشته‌ای با دو رقم تبدیل می‌کند (مثلاً «۳» → «03»). الگو: تقسیم و باقیمانده برای جدا کردن ساعت، دقیقه و ثانیه.